Technical Report: Pepper Medical Assistant

Comprehensive System Analysis and Deployment Implementation

Arab Academy for Science and Technology and Maritime Transport (AASTMT) College of Artificial Intelligence - Graduation Project 2025/2026 Authors: Ahmed El-Shaer, Aly Lotfy, Abdulrahman Karam, Amr Emara, Amira El-Sayed, Mohammed Maghwary, Youssef Ahmed Tawfik

+2

Table of Contents

1. Executive Summary
2. Introduction & Motivation
3. System Architecture Overview
4. Tier 1-Robot Interface Layer
5. Tier 2-Backend Server Layer
6. Tier 3-User Interface Layer
7. Database Design & Data Management
8. Voice & Speech Processing Pipeline
9. AI Integration - Whisper, Gemini & RAG
10. Navigation System
11. Advanced Triage Logic System
12. Security, Authentication & RBAC
13. Inter-Process Communication Mechanisms
14. Results & Evaluation
15. Current Work Status
16. Future Work & Roadmap
17. Conclusion

1. Executive Summary

The Pepper Medical Assistant is an intelligent, humanoid hospital kiosk system developed as a graduation project at AASTMT's College of Artificial Intelligence. It deploys SoftBank Robotics' Pepper robot as a patient-facing medical assistant at Andalusia Hospital, integrating cutting-edge Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), automatic speech recognition, and autonomous navigation into a single unified platform. The system is structured across three tiers: a legacy Python 2.7 NAOqi robot layer, a Python 3 Flask backend with AI services, and an HTML/CSS/JavaScript tablet web application. Key technical achievements include a measured end-to-end AI response latency of 2.23 seconds, a

Word Error Rate (WER) of 9.97% using fine-tuned Whisper, and a 100% API success rate during testing. The most recent development milestone - the subject of this report - is the implementation of an Advanced Triage Logic System inspired by the Manchester Triage Scale, enabling automated patient symptom assessment, urgency classification, department routing, and real-time staff alerting.

+4

2. Introduction & Motivation

2.1 Problem Statement

Global healthcare systems face compounding pressures: an aging population, chronic clinical staff shortages, increased infection-control requirements following the COVID-19 pandemic, and rising patient volumes in outpatient settings. Hospital reception and triage functions—which require sustained human attention, rapid decision-making, and bilingual communication—are particularly strained. Traditional digital kiosks address queue management but cannot engage patients naturally, assess urgency, or physically escort patients to their destinations. A socially assistive robot equipped with conversational AI offers a scalable, infection-safe, always-available assistant that can welcome, inform, triage, and guide patients without replacing clinical staff.

+4

2.2 Why Pepper

Pepper (SoftBank Robotics, 2014) was selected for its humanoid embodiment, which produces measurably higher patient engagement than screen-only interfaces. Studies confirm 93.8% eye contact maintenance and sub-2-second initial engagement rates with Pepper in pediatric settings. Its physical presence and animated gestures reduce patient anxiety and increase trust, particularly critical when delivering triage outcomes.

+3

2.3 Research Gap

Prior Pepper deployments in healthcare used scripted dialogue trees that collapsed under natural language variation and failed in bilingual environments. While LLM integration improved interaction success rates, existing systems lacked grounded hospital data. Our system addresses this gap through a FAISS-backed RAG pipeline, rule-based triage logic, and a real-time WebSocket architecture bridging the Python 2.7/3 interoperability barrier.

+3

3. System Architecture Overview

The Pepper Medical Assistant employs a three-tier centralized architecture where the robot acts as the physical interface and all computational intelligence is offloaded to an external backend server. This design directly compensates for Pepper's onboard CPU limitations (Intel Atom Z530, 1.6 GHz) and enables the use of modern AI libraries incompatible with the robot's embedded Linux/Python 2.7 environment.

+1

- TIER 1: PEPPER ROBOT LAYER
- NAOqi SDK (Python 2.7) ALTextToSpeech | ALAudio Recorder ALNavigation ALMotion ALRobotPosture ALBasicAwareness Chest Tablet (WebKit) |
- 4-Mic Array (16 kHz mono WAV)
- WebSocket (port 8765) + PSCP file transfer File-Flag IPC (voice_start.flag)
- ▼
- TIER 2: BACKEND SERVER LAYER
- Flask (Python 3) SQLAlchemy + SQLite Whisper STT Gemini 2.5 Flash API FAISS Vector Store |
- RAG Pipeline Triage Logic Engine WebSocket Bridge Navigation Manager
- HTTP (port 8080) WebSocket (port 8765)
- TIER 3: USER INTERFACE LAYER
- HTML5/CSS3/ES5 JavaScript | Tablet Web App (on Pepper) i18n.js (AR/EN) | demo_store.js (localStorage) |
- WebSocket 12 Screens: Home, Book, Schedule, Triage, Emergency, etc.

The architecture follows an API-first design principle: Pepper acts purely as a sensor (microphone, camera) and actuator (TTS, movement, display) while the backend server provides all intelligence, persistence, and business logic. This enables the system to be upgraded without any modifications to the robot hardware or firmware.

+1

4. Tier 1 - Robot Interface Layer

4.1 NAOqi Framework

The robot layer runs on Pepper's embedded Linux system using NAOqi OS and is controlled exclusively through the NAOqi Python SDK 2.7. All robot modules are accessed via ALProxy remote procedure calls as detailed in the technical documentation.
+2

Module	Purpose
ALTextToSpeed	Verbal output in English and Arabic
ALAudioRecord	Captures 4-second WAV audio at 16,000 Hz mono
ALNavigation	SLAM-based navigation (navigate To InMap with obstacle avoidance
ALMotion	Fallback relative movement (moveTo) when ND SLAM map exists
ALRobotPostur	Manages standing/sitting postures
ALBasicAwareness	Autonomous engagement (paused during navigation)
ALAutonomousLife	High-level behavior management (disabled at startup)
ALBattery	Battery monitoring with low-charge warnings

4.2 Audio Capture Pipeline

The NAOqi ALAudioRecorder module captures a 4-second audio recording from the robot's front head microphone. The recording parameters are optimized for high-fidelity transcription processing.
+2

Format: WAV (uncompressed PCM)
Sample Rate: 16,000 Hz (optimized for Whisper STT)
Channels: 1 (mono)

- Duration: 4 seconds per interaction
- Storage path: /tmp/voice.wav on robot filesystem

The choice of 16 kHz mono was deliberate: it minimizes file size for faster PSCP transfer while matching Whisper's native training sample rate, eliminating internal resampling and improving transcription accuracy.

4.3 Python 2.7/3 Interoperability

A critical engineering challenge is that NAOqi requires Python 2.7, while modern AI libraries require Python 3. The system resolves this through an API-first design, UTF-8 explicit encoding, and filesystem-level IPC mechanisms.
+1

- 1. API-first design: All AI processing occurs server-side, the robot script only makes HTTP requests
- 2. UTF-8 explicit encoding: All TTS strings are explicitly encoded reply.encode("utf-8") before passing to ALTextToSpeech.say() preventing UnicodeDecodeError crashes on Arabic text
- 3. File-flag IPC: A filesystem-level communication channel that works across Python version boundaries without any network dependency (detailed in Section 13)

4.4 Tablet Display

Pepper's 10.1" chest tablet runs a WebKit-based browser rendering the web application. JavaScript conforms to strict ES5 standards to maintain compatibility with the legacy browser engine.
+1

5. Tier 2 - Backend Server Layer

5.1 Flask Application

The backend is a Flask application running on Python 3, serving on port 8080. It handles all HTTP API requests from the tablet, manages the SQLite database, and orchestrates the voice processing pipeline.
+4

5.2 Complete API Endpoint Reference

Method	Endpoint	Description
GET	/	Serves index.html
POST	/api/login	Patient/staff authentication
POST	/api/signup	New user registration
GET	/api/me	Session status check
POST	/api/logout	Session invalidation
POST	/api/process_audio	Whisper transcription + Gemini reply
POST	/api/chat_ai	Text-based Gemini chat with history
GET	/api/ai_health_tips	Personalized health tips via Gemini
POST	/api/triage_assess	[NEW] Manchester Triage Scale assessment

GET	/api/my_appointments	Patient's appointment history
GET	/api/doctors	All doctors list
GET	/api/departments	All unique specialties
GET	/api/doctors_by_department/<name>	Doctors filtered by specialty
GET	/api/schedule/int:doctor_id	Doctor's availability schedule
GET	/api/navigation_targets	Navigation room coordinates
GET	/static/qimessaging.js	NAOqi tablet JS bridge

5.3 WebSocket Bridge

A standalone asyncio WebSocket server acts as a message router between all connected clients. It implements a broadcast model to facilitate real-time communication between the tablet, navigation manager, and voice modules.
+2

- Tablet Robot start_recording stop_recording navigate say
- Robot Triage Tablet: result (transcription + AI reply), nav complete nav_failed
- Robot say (emergency TTS alert for Level 1 patients)

The server logs all non-ping message types for debugging and gracefully handles disconnections with reference set cleanup.

5.4 Navigation Manager

The navigation manager is a utility module that provides coordinate retrieval and target management. It serves a 40-target room directory including target names, specialties, and precise spatial coordinates.

+3

6. Tier 3 - User Interface Layer

The UI layer consists of 12 HTML pages served as static files from Flask's static directory. All pages are ES5-compliant, responsive at 1280x800 (Pepper's tablet resolution), and support full Arabic/English switching via 118n. js. The design language is consistent: tan/brown palette (#b89a6c primary, #5a4a3b text, #f7f4f0 background), 24px border-radius cards, and touch-optimized button sizing (minimum 52px tap targets).

+2

6.1 Page Inventory

Page	File	Description
Home	index.html	Main navigation hub, 8 tiles across 2 pages, voice overlay; auth status, appointment highlight badge
Symptom Check	triage.html	[NEW] 4-step triage wizard with AI assessment
Book Appointment	book.html	Department Doctor Date/Time booking flow
Check Schedule	schedule.html	Browse doctor availability
My Appointments	appointments.html	Patient's personal appointment history

Emergency	emergency.html	2-second hold-to-confirm emergency alert with WebSocket TTS
Guide to Room	guide.html	Dynamic navigation target list WebSocket navigation dispatch
Health Tips	tips.html	Gemini-generated personalized health advice
Chat/FAQ	chat.html	Real-time Gemini chat with history; local fallback replies
Staff Dashboard	staff dashboard.html	Appointments, Emergencies, Doctors, Schedules, Patients, [NEW] Triage tabs
Login	login.html	Patient/staff login with role selector
Sign Up	signup.html	New patient/staff registration

6.2 Shared JavaScript Modules

118n.js-Internationalization engine. Implements a translation dictionary with 80+ keys covering all pages in English and Arabic. Provides 118N.t(key), I18N.getLang(), 118N.toggleLang(), and 118N.applyI18n() for DOM-wide text substitution. Language preference persists in localStorage. The engine uses data-i18n HTML attributes to mark

translatable elements. RTL layout (dir="rtl") is applied to the <html> element when Arabic is selected, triggering CSS mirror rules throughout the UI.

+4

demo_store.js - Client-side state management using localStorage, Implements the window.STORE API with the following subsystems:

- Subsystem: Authentication
- Storage Key: pepper_session
- Methods: login() logout() me() signup()
- Subsystem: Appointments
- Storage Key: pepper_appointment
- Methods: book() myAppointments() allAppointments()
- Subsystem: Doctors
- Storage Key: pepper_doctors
- Methods: getDoctors() addDoctor() removeDoctor()
- Subsystem: Schedules
- Storage Key: pepper_schedules
- Methods: getSchedule() setSchedule()
- Subsystem: Patients (derived)

  +1
- Storage Key: pepper_patients
- Methods: getPatients()
- Subsystem: Emergencies
- Storage Key: pepper_emergencies
- Methods: addEmergency() listEmergencies clear Emergencies()
- Subsystem: [NEW] Triage
- Storage Key: pepper_triages
- Methods: addTriage() listTriages() clear Triages()

The store is seeded with 25 patient accounts, 5 staff accounts, and 80 doctors across 20+ specialties on first load, enabling full offline demonstration without a database.

7. Database Design & Data Management

7.1 SQLAlchemy Models

The Flask backend uses SQLAlchemy with a SQLite database (app db). The schema defines 7 models:

+4

- Doctor: id (PK), name, specialty, title, branches
  +4
- Schedule: id (PK), doctor_id (FK), day_of_week, start_time, end_time
  +4
- Appointment: id (PK), doctor_id (FK), patient name, appointment_date, time_slot
  +4
- Branch: id (PK), name, address, city, notes
  +2
- Department: id (PK), name, description
  +2
- Contact: id (PK), type, details
  +4
- Patient: id (PK), name, case_number, password
  +4
- Staff: id (PK), name, role, password
  +4

7.2 Knowledge Base Statistics The hospital knowledge base contains data across 8 Andalusia Hospital branches spanning Alexandria, Cairo, and Giza:

+1

Metric	Value
Total Doctors	80
Medical Specialties	22
Navigation Targets	40 rooms
Hospital Branches	8
Default Schedule Slots per Doctor	2
Seed Patient Accounts	25

Seed Staff Accounts	5

Data from:

Cardiology is the most represented specialty (10 doctors), followed by Dermatology (8) and Dentistry (7), reflecting Andalusia Hospital's clinical focus areas.

7.3 Database Initialization

+3

7.4 Data Architecture Layers

+1

- Visible Service Layer: Doctor directories, department information, schedules accessible to all users; served through the RAG pipeline
- Internal Operational Layer. Patient records, appointment history, emergency logs, triage assessments - restricted to authenticated users by session and role checks

8. Voice & Speech Processing Pipeline

8.1 End-to-End Flow

+1

1. Patient taps "Tap to Speak" on tablet
2. Tablet sends WebSocket message: {type: "start_recording", lang: "en"}
3. ws_bridge.py broadcasts to all clients nav_bridge.py receives it
4. nav_bridge.py writes voice_start flag to filesystem
5. MainVoice.py (Python 2.7) detects flag file deletes it
6. Pepper says "Please speak now." via ALText To Speech
7. ALAudio Recorder records 4 seconds /tmp/voice.wav on robot
8. PSCP pulls file from robot to backend PC local voice wav
9. MainVoice.py HTTP POSTs voice.wav to /api/process_audio
10. Whisper transcribes audio detected text + language
11. Gemini 2.5 Flash generates contextual reply

12. Backend returns (text, reply, lang) to MainVoice.py
13. MainVoice.py encodes reply UTF-8 tts.say(reply)
14. Pepper speaks the reply aloud
15. ws_bridge.py also broadcasts (type:"result",...) to tablet
16. Tablet voice overlay displays transcription + AI reply

8.2 File-Flag IPC Mechanism

+2

- Writer: nav bridge.py (Python 3) creates the flag file
- Reader/Consumer: MainVoice.py (Python 2.7) polls every 500ms, detects the file, immediately deletes it (preventing double-trigger), then begins recording

A companion file lang flag carries the language preference ("en" or "ar") from the tablet UI to MainVoice.py, enabling language-specific TTS announcements and correct Whisper language detection.

8.3 PSCP File Transfer

The SSH password nao is Pepper's default factory credential. The transfer typically completes in under 200ms for a 4-second 16kHz mono WAV file (~128 KB).

+3

8.4 Whisper Speech-to-Text

+3

9. AI Integration - Whisper, Gemini & RAG

9.1 Gemini 2.5 Flash Integration

+4
Measured performance:

- Average latency: 2.23 seconds (range: 1.43s-2.88s)
- API success rate: 100% during testing
- Language auto-detection: correct language response on Arabic queries without explicit language parameter

9.2 RAG Pipeline

+1

- User Query
- Text Embedding (sentence-transformers)
- FAISS Vector Store
- (Hospital Knowledge Base: Doctors, Departments, Branches, Contacts)
- Semantic Search Top-K relevant passages
- Context-Augmented Gemini Prompt
- Grounded Response

The FAISS index is built from the structured CSV datasets, encoding each doctor record, department description, branch location, and contact number as a vector embedding. At query time, the top-K nearest passages are injected into the Gemini prompt as grounding context, ensuring that answers like "Where is the Cardiology department?" return the verified room number from the database rather than a plausible-but-fabricated response.

+2

9.3 AI Health Tips

+4

10. Navigation System

10.1 Architecture

+4

10.2 Dual Navigation Modes

+2

1. LoadExploration(map_path) - loads binary map into memory
2. relocalizeInMap([0, 0, 0]) assumes robot starts at map origin
3. startLocalization() -begins background LIDAR/sonar comparison loop Error codes from SLAM: θ=0K (arrived), 1=KO (path blocked), 2=1 Constraint KO (spatial constraints failed) Mode 2 - Dead Reckoning (Fallback): When no SLAM map exists, ALMotion.moveTo(x, y. 8) executes a direct relative movement command. External collision protection is temporarily disabled to prevent sonar

false-positives from furniture interrupting the movement, then re-enabled on arrival.

+3

10.3 Thread Safety A critical design decision is executing navigateToInMap ()-a blocking call that may take 20-60+ seconds - inside a daemon threading.Thread rather than the main WebSocket loop. Running it on the main thread would block the asyncio event loop, causing the WebSocket server to miss PING/PONG heartbeats and terminate the connection mid-navigation. The is_navigating flag prevents overlapping navigation requests.

+3

10.4 Navigation Target Registry navigation_targets.json stores 40 room targets with the schema: 1 id: 1 "name": "Dr. Ahmed Salem", "specialty": "Cardiology", "room_name": "Room 101-First Floor" "coordinates: [1.5, 1.0, 1.0] Coordinates represent metres (X, Y) and radians ( ) relative to the robot's startup position. The nav_manager.py module provides add_target() for programmatic registration of new rooms without manual JSON editing.

+4

10.5 Status Feedback Loop Upon completing or failing navigation, send_nav_status() transmits a JSON result through the WebSocket to guide.html, which updates its color-coded status bar:

+1

State	CSS Class	Color	Message
Connecting	status-connecting	Ten	"Connecting to robot..."
Connected	status-connected	Green	"Select a doctor"
Navigating	status-navigating	Orange	"Pepper is guiding you..."
Arrived	status-ok	Green	"Arrived at Room X"

Failed	status-error	Red	"Navigation blocked"
Offline	status-offline	Grey	"Robot offline"

Data from:

11. Advanced Triage Logic System

11.1 Overview & Clinical Basis

+4

11.2 Four-Level Classification

Level	Label	Color	Trigger Criteria	Target Response
1	IMMEDIATE	Red	L1 danger sign present OR pain score >= 9	Immediate: staff alerted now
2	VERY URGENT	Orange	Pain >= 7, OR multiple danger signs, OR chest pain >= 5	Within 10 minutes
3	URGENT	Yellow	Pain >= 5, OR 1+ danger sign present	Within 30 minutes
4	STANDARD	Green	Pain <= 4, no danger signs	Routine appointment

---

Data from:

11.3 Patient-Facing Wizard

+3

(Note: The following section mirrors an alternative or extended version of the report's text provided in the source).

Pepper Medical Assistance Robot - Comprehensive Technical Report

Project Title: Integrating Large Language Models and Retrieval-Augmented Generation for Telepresence in Socially Assistive Robots Institution: Arab Academy for Science and Technology and Maritime Transport (AASTMT), College of Artificial Intelligence, Alexandria, Egypt Team: Ahmed El-Shaer, Aly Lotfy, Abdulrahman Karam, Amr Emara, Amira El-Sayed, Mohammed Maghwary, Youssef Ahmed Tawfik

+2

1. Executive Summary This report documents the complete design, implementation, and evaluation of the Pepper Medical Assistance Robot - a graduation project at AASTMT that deploys a Pepper humanoid robot as an intelligent patient-facing assistant inside Andalusia Hospital. The system bridges Pepper's legacy NAOqi SDK (Python 2.7) with a modern Python 3 backend hosting Whisper speech-to-text, the Gemini 2.5 Flash large language model, a Retrieval-Augmented Generation (RAG) pipeline backed by a FAISS vector store, a relational hospital database, a physical navigation engine with SLAM map integration, and a fully bilingual (Arabic/English) tablet web application. Real-world testing recorded an end-to-end AI latency of 2.23 seconds, a Word Error Rate (WER) of 9.97%, and a 100% API success rate during structured trials. The most recent addition to the system is an advanced rule-based and AI-augmented triage logic module that assesses patient symptoms in real time, classifies urgency across four Manchester Triage Scale levels, and routes patients to the correct clinical resource while simultaneously alerting medical staff through the WebSocket bridge and the Pepper robot's text-to-speech system.

+4

2. Introduction and Motivation Healthcare systems worldwide are under unprecedented pressure from aging populations, chronic staffing shortages, and post-pandemic hygiene requirements. Intelligent reception robots offer a pathway to reduce the administrative burden on clinical staff, improve patient routing efficiency, and provide an engaging human-scale interface that is more comfortable than a static information kiosk. Pepper, manufactured by SoftBank Robotics, was selected as the deployment platform

because of its humanoid form factor, built-in 4-microphone array, chest-mounted HD tablet, and established track record in social interaction research. However, Pepper presents a fundamental engineering challenge: its onboard NAOqi framework runs Python 2.7 on a Linux environment that predates modern AI ecosystems by nearly a decade. Whisper requires Python 3.8+, Gemini requires HTTP REST access, and vector search libraries such as FAISS have no Python 2 support. The central contribution of this project is the architectural solution to this interoperability gap: a three-tier system that keeps the robot as a thin physical interface while pushing all AI computation to an external Python 3 backend, connected via a WebSocket relay server and a file-flag IPC mechanism. The clinical deployment context - Andalusia Hospital, Alexandria - provided concrete requirements: the system must support Arabic and English, manage appointment booking and doctor schedules, guide patients physically to specific rooms, perform real-time symptom triage, handle emergencies, and present all functionality through a touch-friendly tablet interface suitable for elderly and pediatric patients.

+4

3. System Architecture **3.1 Overview: Three-Tier Design** The system is divided into three discrete processing tiers, each isolated from the others by well-defined interfaces:

+2

- TIER 1-PEPPER ROBOT LAYER (Python 2.7/NAOqi)
    - ALAudio Recorder (mic input, 16 kHz mono WAV)
    - ALText To Speech (bilingual voice output, Arabic/English) ALMotion/ALNavigation (movement, SLAM navigation)
    - ALRobotPosture (stand/rest postures)
    - Tablet display (opens web app URL via ALTabletService) File-Flag IPC: polls for voice_start.flag every 0.5s
- PSCP audio transfer + WebSocket
- TIER 2-BACKEND SERVER LAYER (Python 3 / Flask) Flask REST API (port 8080) - all business logic SQLAlchemy ORM- SQLite database (Doctors, Schedules, etc.) Whisper ASR - transcribes uploaded WAV audio files Gemini 2.5 Flash - NLU and response generation RAG Pipeline - FAISS vector store + semantic search Triage Engine rule-based + AI-augmented symptom scoring * WebSocket Bridge (port 8765) - real-time relay server
- HTTP (REST) + WebSocket
- TIER 3-USER INTERFACE LAYER (HTML5/CSS3/ES5JS) Tablet web app rendered in Pepper's embedded Chromium 14 pages covering all patient and staff workflows Bilingual (Arabic/English) with 118N.js runtime translation LocalStorage-based demo store for offline resilience Real-time status via persistent WebSocket connection

3.2 Network Topology
The system is deployed on a private hospital Wi-Fi network. The backend PC and Pepper robot share the same LAN. Network addresses are configured in a central config.json file at project root and injected into all processes as environment variables by the master launcher main.py. Default values are:

+3

Parameter	Default Value	Purpose
ROBOT IP	192.168.1.100	Pepper's IP address on LAN
ROBOT PORT	9559	NAOqi broker port
SERVER IP	192.168.1.50	Backend PC IP
SERVER PORT	8080	Flask HTTP API port
WS_PORT	8765	WebSocket bridge port

Data from:

3.3 Inter-Process Communication (IPC) Strategy Two complementary IPC mechanisms bridge the Python 2.7/3.x boundary:

+1

- File-Flag IPC: The tablet UI triggers a voice interaction by writing a voice_start. flag file to the pepper_voice/directory. MainVoice.py (Python 2.7, running on the robot side) polls for this file every 0.5 seconds. Upon detection, it removes the flag, records audio, transfers it via PSCP, and sends it to the Flask backend. A companion lang. flag file carries the user's language preference (en or ar) so the TTS engine uses the correct language for the AI's spoken reply. This file-based mechanism elegantly sidesteps the impossibility of direct Python 2/3 sockets between the NAOqi runtime and modern asyncio code.
  +4

- WebSocket Bridge (ws_bridge.py): A Python 3 asyncio WebSocket server (port 8765) acts as a message bus connecting the tablet UI, the navigation bridge (nav_bridge.py), and the voice module. When the UI sends {"type": "navigate", "target" ...], the bridge broadcasts to all connected clients, including nav_bridge.py which executes the movement command. Similarly, emergency alerts, TTS commands, and voice recording triggers all route through this bus. The server maintains a connected_clients set and broadcasts each message to all peers except the sender, implementing a simple broadcast topology appropriate for the small fixed number of participants.
  +3

4. Tier 1 - Robot Layer **4.1 MainVoice.py (Python 2.7 - Voice Interaction Controller)** MainVoice.py is the primary Python 2.7 script that runs continuously on (or alongside) the robot. Its responsibilities are:

+2

1. Flag polling loop - checks for voice_start.flag at 500ms intervals using os.path.exists()
2. Language selection - reads lang. flag to determine whether to speak Arabic or English.
3. TTS announcement uses ALTextToSpeech.setLanguage() followed by tts.say() to tell the patient to speak. Arabic strings are explicitly encoded as LITF-a byte strings before passing to the legacy NAOqI API to prevent UnicodeDecodeError crashes.
   +1
4. Audio recording calls ALAudioRecorder.startMicrophones Recording) with parameters: path/tmp/voice.wav format wav, sample rate 16,000 Hz channel mask (0,0,1,0) (front microphone). Records for 4 seconds then calls stopMicrophones Recording()
   +1
5. PSCP file transfer executes a system call to pscp.exe (PuTTY Secure Copy) to pull the WAV file from the robot's/tmp/voice.wav to the backend PC's working directory using SSH credentials (nao@{ROBOT_IP)
6. Backend submission -HTTP POSTs the WAV file to Vapi/process_audio as multipart form data.
7. Reply utterance - receives the AI reply text, re-encodes it to UTF-8 if it is a unicode object, sets the TTS language again, and calls tts.say (reply)

All IPs and ports are read from environment variables injected by main.py, making the script hardware-agnostic.

4.2 nav_bridge.py (Python 2.7 - Navigation & TTS Bridge)
nav_bridge.py is a second critical robot-side script. It serves as the WebSocket client that connects Pepper's physical movement capabilities to the backend message bus. Its design is notable for several engineering decisions made to ensure robustness in a clinical environment:

+2

- Hardware Initialization Sequence:
- 1. Connect ALText To Speech ALMotion ALNavigation ALRobotPosture ALBattery proxies.
- 2. Disable ALAutonomousLife (prevents the robot's default "life" behaviors from interrupting navigation).
- 3, Pause AL BasicAwareness (prevents the head-tracking system from overriding motion commands).
- 4. Call motion.wakeUp and posture.goToPosture ("Stand", 1.0) to ensure the robot is upright.
- 5. Attempt to lead prototype_room.explo a pre-recorded SLAM map generated by map_exploration, py If the map exists, call loadExploration() relocalizeInMap ([0,0,0]) and startLocalization() to engage full obstacle-avoidance navigation.
- Dual Navigation Mode:
- SLAM Mode (primary): Uses ALNavigation.navigateToInMap([x, y, the tal for path planning with obstacle avoidance. Retums error codes: 0 (success), 1 (path blocked), 2 (spatial constraint unmet).
- Fallback Mode. Uses ALMotion.moveTo(x, y, theta) for direct relative movement when no SLAM map is available. Before moveTo external collision protection is disabled to prevent sonar false-positives from furniture halting movement in demonstration rooms.

  +1
- Daemon Thread Pattern: navigateToInMap() is a blocking call that can take 20-60+ seconds. Executing it on the main WebSocket thread would block the asyncio heartbeat loop, causing the server to drop the connection. The solution is to dispatch each navigation command to a threading.Thread(daemon=True), keeping the main loop free to process PING/PONG keep-alives and incoming say commands during navigation.

  +2
- Message Routing: The bridge handles four message types:
- navigate dispatches navigation thread
- say direct TTS utterance (used by emergency.html)
- start_recording writes voice_start.flag and lang. flag to trigger MainVoice.py
- All others logged and ignored

- Battery Monitoring: Before each move To, the bridge reads ALBattery.getBatteryCharge() and logs a warning if charge is below 30%, as low battery causes inaccurate movement distances on Pepper hardware.

4.3 Supporting Robot Scripts The Scripts/directory contains a suite of utility and diagnostic tools:

+1

Script	Purpose
pepper_diagnostic.py	Connects to all NAOqi services and reports hardware health
pepper_selftest.py	Runs a sequence of TTS, motion, and sensor tests
pepper_stand up.py	Forces the robot to the Stand posture
pepper_move_arm.py	Test script for arm joint angle control
Turn 180.py	Rotates the robot 180 degrees (used for demonstration)
move_pepper.py	Manual movement controller
benchmark_gemini.py	Measures Gemini API latency over multiple calls
pepper_record.py	Standalone voice recording test
pepper_talk.py	Standalone TTS test
test_services.py	Validates all Flask API endpoints
test_tts.py	Tests Arabic/English TTS output

camera/server.py	MJPEG camera stream server
camera/bridge.py	Camera WebSocket bridge

Data from:

---

5. Tier 2 - Backend Server Layer **5.1 Flask Application (app.py)**
The Flask backend is the system's computational core. It is instantiated with a dynamically resolved static_folder that searches both the app/ directory and its parent for the HTML/CSS/JS files, ensuring the application runs correctly whether launched from main.py or directly. Key configurations:

+3

- Cache busting: Every HTTP response has Cache-Control: no-cache, no-store, must-revalidate to ensure Pepper's Chromium tablet always loads the latest version of static files.
- Secret key: Session management uses Flask's session object for authenticated user state across requests.
- SQLAlchemy: Configured with sqlite:///app.db relative to the app/directory. SQLALCHEMY TRACK_MODIFICATIONS False to suppress deprecation warnings.
- Whisper initialization: The Whisper model (base- multilingual) is loaded once at server startup to avoid per-request cold-start overhead. The multilingual base model was selected over base.en because the system supports Arabic speech input.
  +1

5.2 Database Design The database uses SQLite via SQLAlchemy ORM and is automatically created and seeded on first run. The schema comprises eight models:

+2
Core Medical Entities

- Doctor Stores all physician records loaded from Doctors.csv. Fields: id (PK), name, specialty, title, branches. The specialty field is used as the department identifier throughout the system. Has one-to-many relationships with Schedule and Appointment.
  +1

- Schedule - Models a doctor's weekly availability. Fields: id, doctor_id (FK), day_of_week (0=Monday...6=Sunday), start_time, end_time. The default seeding creates two schedule slots per doctor (Monday 09:00-13:00 and Wednesday 14:00-17:00).
  +1
- Appointment - Records a booked appointment. Fields: id, doctor_id (FK), patient_name, appointment_date, time_slot. Patient name is stored as a string (not FK to Patient) to allow guest bookings.
  +1
- Branch - Represents a physical hospital branch. Fields: id, name, address, city, notes. Seeded from Branches.csv covering 8 branches across Alexandria, Cairo, and Giza.
  +1
- Department - Stores department/service names and descriptions. Fields: id, name, description. Seeded from Departments.csv covering 16 unique departments.
- Contact - Emergency and departmental contact numbers. Fields: id, type, details. Seeded from Contacts.csv.

User Management

- Patient - Registered patients. Fields: id (user-supplied integer), name, case_number (medical case description e.g. "Hypertension"), password (plain text - see Section 10 for security discussion). The case_number field feeds the personalized AI health tips generator.
  +1
- Staff Hospital staff accounts. Fields: id, name, role, password. Staff accounts have access to the staff dashboard which exposes appointment management, emergency alerts, triage records, doctor management, and patient records.

Seeding Strategy
The setup_database() function checks whether any records exist before seeding, making it idempotent on subsequent launches. Seeding sources:

+1

- Branches, Departments, Contacts - Read from CSV files located either in app/or the parent server/directory.
- Doctors-Loaded from Doctors.csv rows where name contains "Clinic" are skipped (clinic entries in the CSV, not individual doctors).
- Default Schedules - Two slots created per doctor automatically.

- Dummy Users - One patient (Ahmed Al ID 101) and one staff member (Nurse Mona) created for demonstration.

5.3 API Endpoints The Flask application exposes the following REST endpoints:

+1
Authentication
Endpoint	Method	Description
/api/login	POST	Validates role, id, password against DB; creates Flask session
/api/logout	POST	Clears Flask session
/api/me	GET	Returns current session user name and login status
/api/signup	POST	Creates Patient or Staff record; rejects duplicate IDs
Data from:

+1
Medical Information
Endpoint	Method	Description
/api/doctors	GET	Returns all doctors {name, specialty}]
/api/departments	GET	Returns distinct specialties from Doctor table
/api/doctors by department/<name>	GET	Filters doctors by specialty
/api/schedule/<doctor_id>	GET	Returns all Schedule rows for a doctor with human-readable day names
/api/my_appointments	CET	Returns all Appointments for the logged-in patient
Data from:

+1
AI Services
Endpoint	Method	Description
/api/process_audio	POST	Accepts WAV file, runs Whisper ASR, sends transcription to Gemini, returns (text, reply, lang)
/api/chat_ai	POST	Accepts message, history, lang) queries Gemini with conversation history
/api/ai_health_tips	GET	Generates 4 personalized health tips for logged-in patient using Gemini + patient case number
/api/triage_assess	POST	New Rule-based + AI-augmented triage assessment (detailed in Section 8)
Data from:

+1
Navigation

Endpoint	Method	Description
/api/navigation_targets	GET	Reads navigation_targets.json and returns target list to the guide page

Data from:

5.4 Gemini Integration (interact_with_gemini)
The interact_with_gemini() helper function constructs requests to the Gemini 2.5 Flash API. Key design decisions:

+1

- System Prompt: Injected as a separate systemInstruction field (not prepended to message history) to cleanly separate persona instructions from conversation content. The system prompt differs by language. English instructs brevity under 40 words, Arabic instructs responses under 40 Arabic words.
  +1
- Personalization: The patient's name is included in the system prompt to enable Germini to address the patient directly.
- Conversation History: The history list parameter carries Gemini's expected message format [{"role": "user" | "model", "parts": [{"text": "..."1 for multi-turn conversations. The chat page maintains and sends this history on each request.
  +1
- Error Handling: If candidates is absent from the API response (quota exceeded, content filtered), a graceful fallback string is returned.

5.5 WebSocket Bridge (ws_bridge.py) The WebSocket relay server is a compact Python 3 asyncio application built on the websockets library. It operates as a pure broadcast relay: every message received from one client is forwarded to all other connected clients. This design requires no knowledge of message types or routing rules at the bridge level - all routing intelligence resides in the sender and receiver scripts. Key implementation details:

+4

- Connection management: A module-level connected_clients set tracks all active websocket objects. Clients are added on connection and removed in a finally block to handle both graceful disconnection and unexpected drops.
  +1
- Message filtering: ping pong and hello messages are suppressed from console output to reduce log noise during normal operation.

- Port configurability: Reads WS PORT from environment variable with default 8765 Graceful shutdown: KeyboardInterrupt breaks the main coroutine cleanly.

6. Navigation System **6.1 Navigation Targets (navigation_targets.json)**
The navigation target registry stores 40 room destinations for Andalusia Hospital. Each target entry contains: id: 1 "name": "Dr. Ahmed Salem", "specialty": "Cardiology", "room_name": "Room 101-First Floor" "coordinates": [1.5, 1.0, 1.0] Coordinates are [x_meters, y_meters, theta_radians] relative to the robot's start position (map origin). The file covers all 4 floors: Ground Floor (Reception, Emergency, Radiology, Dentistry, Physiotherapy), First Floor (Cardiology, Internal Medicine, Dermatology, Ophthalmology, Audiology, Clinical Nutrition), Second Floor (Orthopedics, Pediatrics, ENT, Pulmonology, OB/GYN, Vascular Surgery, Gastroenterology, Urology, General Surgery), Third Floor (Neurology, ENT, Nephrology, Hepatology, Endocrinology, Rheumatology, Hematology), and Fourth Floor (Clinical Oncology, Neurosurgery, Psychiatry, Plastic Surgery, Sleep Medicine). The file includes a "prototype_mode": true flag indicating that coordinates are currently set for a demonstration room configuration and must be recalibrated via SLAM mapping for the actual hospital deployment.

+4
6.2 Navigation Manager (nav_manager.py) A Python 3 utility module that provides two functions:

+1

- get_navigation_targets() - reads and returns the targets list; used by app.py to serve the /api/navigation_targets endpoint.
- add_target(name, specialty, room_name, x, y, theta) - programmatically appends a new target to the JSON file; useful during the initial room mapping phase.
  +1

6.3 SLAM Map Generation (map_exploration.py) A NAOqi Python 2.7 script that drives Pepper through a hospital floor in autonomous exploration mode, building a 2D occupancy grid map using the robot's laser range finders and ultrasonic sensors. The resulting.explo binary file is saved to navigation/prototype_room.explo and loaded at runtime by nav_bridge.py.

+2
6.4 End-to-End Navigation Flow

1. Patient opens guide, html on the tablet
2. guide.html GETS/api/navigation_targets and renders a button for each doctor.
3. Patient taps a doctor button guide.html sends type": "navigate", "target": [x, y theta], "doctor_name": "...", "room_name": "..." over WebSocket.

4. ws_bridge.py broadcasts to all clients, including nav_bridge.py
5. nav_bridge.py receives the message, validates coordinates, and dispatches a daemon navigation thread
6. Pepper announces the destination via TTS and begins moving.
7. On arrival, nav_bridge.py sends ("type": "nav_complete", "room_name": "back through the bridge.
8. guide.html receives this message and updates the status bar to "Arrived at Room X."

7. Speech and AI Processing Pipeline **7.1 End-to-End Voice Interaction Flow** The voice interaction pipeline involves all three tiers and crosses the Python 2.7/3.x boundary twice: Patient taps Tap to Speak on tablet

+2

- guide/index.html sends (type:"start_recording") over WebSocket
- ws_bridge.py broadcasts
- nav_bridge.py receives
- nav_bridge.py writes lang.flag ("en" or "ar")
- nav_bridge.py writes voice_start.flag (1)
- MainVoice.py (polling loop) detects voice_start.flag
- MainVoice.py: sets TTS language, says "Please speak now MainVoice.py: records 4s via ALAudio Recorder/tmp/voice.wav MainVoice.py: PSCP pulls voice way to backend PC
- MainVoice.py: HTTP POST/api/process_audio (multipart WAV)
- Flask: whisper.transcribe(wav) text, language) Flask interact_with_gemini(text, [], user_name, lang) Flask: returns (text, reply, lang)
- MainVoice.py: sets TTS language, tts.say(reply)
- ws_bridge.py also receives result and forwards to tablet
- tablet voice overlay displays "You said: X / Pepper: Y

Total measured latency: ~0.5s transcription + ~1.8s Gemini generation = 2.23 - 2.5 end-to-end.

7.2 Whisper ASR Configuration
The system uses OpenAI Whisper base (multilingual, 74M parameters). This model was selected over base.en (English-only) specifically to support Arabic speech, which is the primary language of Andalusia Hospital patients. The multilingual base model supports 99 languages and uses a language detection step that identifies Arabic (code ar) vs. English (en), allowing the system to automatically switch the Gemini response language without requiring the patient to manually select a language before speaking. The measured Word Error Rate (WER) of 9.97% was achieved through backend-side audio

preprocessing to reduce hospital background noise. This significantly outperforms BERT-FT (23.1% WER) and Wav2Vec2-base (32.8% WER) on the same clinical audio samples.

+4

7.3 RAG Pipeline The Retrieval-Augmented Generation pipeline grounds Gemini's responses in verified hospital data, preventing hallucination of incorrect department locations, doctor names, or visiting hours. The pipeline:

+2

1. Converts the patient's query into a vector embedding,
2. Performs semantic similarity search against a FAISS vector store containing embedded hospital knowledge (doctor profiles, departments, branches, contact info).
3. Retrieves the top-K most relevant knowledge chunks.
4. Injects retrieved chunks as context into the Gemini system prompt before generating the final response. This ensures that when a patient asks "Where is Dr. Ahmed Salem's room?", the answer is drawn from the structured hospital database rather than generated speculatively.
   +1

8. Advanced Triage Logic System (Current Work) The advanced triage system is the most recently completed major subsystem. It replaces the original single-button emergency page with a structured, multi-step clinical assessment that guides patients through a symptom evaluation and produces an AI-verified urgency classification.

+2

8.1 Clinical Basis: Manchester Triage Scale The triage logic is modeled on the Manchester Triage Scale (MTS), which is the standard used in Emergency Departments across the UK, Egypt, and many other countries. The MTS classifies patients into five levels, of which we implement four applicable to an ambulatory hospital reception context:

+2

Level	Color	MTS Name	Max Wait	Clinical Threshold
1	Red	Immediate	0 min	Life-threatening immediate

				intervention required
2	Orange	Very Urgent	10 min	Serious risk of deterioration
3	Yellow	Urgent	30 min	Significant problem; needs timely assessment
4	Green	Standard	90-120 min	Non-urgent; routine care appropriate

Data from:

8.2 Frontend: triage.html (4-Step Wizard) The triage page is implemented as a single-page multi-step wizard in strict ES5 JavaScript to ensure compatibility with Pepper's embedded Chromium tablet browser (which is based on a 2014-era WebKit engine that does not support ES6 const/let, arrow functions, fetch(), Promises, or template literals). All DOM manipulation uses var, XMLHttpRequest, and string concatenation.

+2

Step 1-Chief Complaint Selection Six large touch-optimized buttons in a 2-column grid (minimum 88px height for reliable touch on the tablet screen): Chest/Heart Breathing Head/Neuro Pain/Injury Fever/Infection Other The selected button receives a visual highlight (tan border + light background) and the selection is stored in the selectedCat state variable. Navigation to Step 2 is blocked with an alert if no category is selected. Step 2-Pain Score (1-10) Ten numbered circle buttons with color-coded severity coding: green for 1-3 ("No Pain"), orange for 4-6 (moderate), red for 7-10 (severe). Clicking a number highlights it and stores the value in painScore. Navigation is blocked if no score is selected.

+4

Step 3-Danger Sign Checklist A dynamic checklist is built based on the category selected in Step 1. Each category has 4 specific clinical danger signs drawn from Manchester Triage discriminators:

+1

- Category: heart
- Danger Signs: Chest tightens/squeezes, Pain spreads to arm or jaw; Sweating heavily Feeling faint
- Category: breathing
- Danger Signs: Cannot speak in full sentences, Lips turning blue; Using neck muscles to breathe; Feeling faint
- Category: neuro
- Danger Signs: Sudden thunderclap headache; Face drooping; Arm weakness/slurred speech; Feeling faint
- Category: pain
- Danger Signs: Deformity/bone visible: Cannot move injured area, Severe open wound; Feeling faint
- Category: fever
- Danger Signs: Neck stiffness; Rash spreading rapidly, Confused behavior; Feeling faint
- Category: other
- Danger Signs: Feeling faint, Chest tightness; Sudden deterioration; Cannot speak in full sentences

Each checklist item is a full-width tappable row. Checked items are stored in the checkedSigns array. No minimum selection is required - the patient may have none of the listed signs.

+1

Step 4-Assessment and Result On pressing "Get Assessment", the wizard POSTS to/api/triage_assess with: 1 "chiefComplaint": "heart" "painScore": 8, "symptoms": ["Pain spreads to arm or jaw", "Sweating heavily"], "lang": "en"
While the HTTP request is in flight, a spinner is shown. On response, the result card is rendered with:

+4

1. A large colored circular badge showing the level number and label.
2. Complaint, pain score, referred department, and any checked danger signs.
3. The AI-generated recommendation text.
4. A level-appropriate action button.

Level-Specific Actions:

- Level 1 (Immediate): Triggers STORE.addEmergency() to record a critical alert in the staff dashboard, sends a WebSocket say message to make Pepper verbally announce the emergency (Emergency alert! Patient reporting chest symptoms with critical danger signs. Immediate assistance required."), and navigates to emergency.html
  +1
- Level 2 (Very Urgent): Navigates to emergency.html
- Level 3 (Urgent): Navigates to book.html for specialist appointment booking..
- Level 4 (Standard): Navigates to tips.html for health advice.
- All triage results are persisted to localStorage via STORE.addTriage() for staff review.

8.3 Backend: /api/triage_assess The backend endpoint implements a two-stage assessment: *Stage 1- Rule-Based Classification (deterministic, always runs):* Two sets of clinical danger signs are defined as Python sets:

+3

- 1 SIGNS (triggers Level 1 or escalates): "Cannot speak in full sentences", "Lips or fingertips turning blue", "Face drooping on one side", "Arm weakness / slurred speech", "Pain spreads to arm or jaw", "Severe open wound / major bleeding", "Feeling faint/ nearly collapsed"
- 12 SIGNS (triggers Level 2 or escalates): "Chest tightens/squeezes , 'Sweating heavily", "Using neck muscles to breathe", "Sudden thunderclap headache", "Neck stiffness", "Rash spreading rapidly", "Confused / unusual behavior", "Severe open wound", "Chest tightness", "Sudden deterioration"

Classification logic:

- if (symptom_set L1_SIGNS is non-empty) OR (pain_score >= 9):
- ->Level 1 IMMEDIATE, Emergency Medicine
- elif (symptom_set L2_SIGNS is non-empty) OR (pain >= 7) OR (chest/breathing AND pain >= 5)
- ->Level 2 VERY URGENT, Emergency Medicine
- elif (pain >= 5) OR (any symptoms checked):
- ->Level 3 URGENT, specialty-mapped department
- else
- ->Level 4 STANDARD, specialty-mapped department

Department mapping for Level 3/4

- heart -> Cardiology
- breathing -> Pulmonology
- neuro -> Neurology
- pain -> Orthopedics
- fever -> Internal Medicine
- other -> Internal Medicine

Stage 2-Gemini AI Enrichment (optional, best-effort): After the rule-based classification determines the level, Gemini 2.5 Flash is queried to generate a warm, personalized recommendation message. The prompt explicitly instructs Gemini not to change the triage level - only to rephrase the recommendation in a reassuring clinical tone. If the Gemini call fails (network error, timeout, quota), the system silently falls back to the pre-written rule-based recommendation text. This ensures the triage system remains fully functional even when offline.

+4
The complete response structure: success true, "level": 2, "label": "VERY URGENT", "color": "Orange "department": "Emergency Medicine
recommendation": "Your condition is serious. Please go directly to Emergency Reception a nurse will assess you within 10 minutes."

+4
8.4 Demo Store: Triage Storage Layer The demo_store.js client-side storage module was extended with a TRIAGES localStorage key and three new public API methods:

+1

- STORE.addTriage(data) - Persists a triage record with timestamp, patient name (from active session or "Guest"), all symptom data, level, label, and recommendation.
- STORE.listTriages() -Returns all stored triage records.
- STORE.clearTriages(Clears all triage records (staff action).
  +1

8.5 Staff Dashboard: Triage Tab The staff dashboard was extended with a fifth "Triage" tab alongside the existing Appointments, Emergency, Doctors, and Patients tabs. The triage panel displays records in reverse chronological order (most recent first) with visual severity coding:

+2

- Levels 1 and 2 use card.danger styling (red border) to immediately draw staff attention.
- Levels 3 and 4 use standard card styling.

- Each record shows: colored level badge (L1-IMMEDIATE, L2-VERY URGENT, etc.), patient name, chief complaint, pain score, any checked danger signs, the AI recommendation, and timestamp.
- The "Clear All" button allows staff to acknowledge and dismiss a batch of triage records.

8.6 Home Screen Integration The "About" tile on the home screen (index.html) was replaced with a "Symptom Check" tile (bilingual: English "Symptom Check" / Arabic "فحص الأعراض" linking to triage.html. This decision reflects the clinical priority hierarchy of the system: triage functionality is more clinically valuable as a home-screen tile than general information about the robot.

+1

9. User Interface Layer **9.1 Design System** All 14 pages share a consistent visual identity:

+2

- Color palette: Primary tan-brown #b89a6c deep brown text #5a4a3b, warm muted #8a7b6b backgrounds #f7f4fe and #ffffff
- Typography: System sans-serif stack (no Google Fonts the tablet has no internet access during deployment; blocking font requests would cause white-screen hangs until timeout).
- Layout: All flexbox layouts include-webkit prefixes for Pepper's older WebKit engine.
- Touch targets: All interactive elements have a minimum height of 44-80px for reliable touch Interaction.
- Animation: CSS transitions and opacity transform animations use -webkit-transition prefixes. Tile entrance animations use staggered setTimeout delays.
- JavaScript: Strict ES5 throughout-no const let arrow functions, fetch() Promises, ford of or template literals.

9.2 Internationalization (i18n.js) The 118N object provides a runtime translation system:

+1

- 118N.t(key, args) Returns the translated string for the current language, with placeholder substitution.
- 118N.getLang() Returns "en" or "ar from localStorage
- 118N.toggleLang - Switches language and calls window.onLangChange() if defined. I18N.applyI18n() translated strings. Scans the DOM for data-118n

attributes and replaces textContent with
+2
- 118N.applyDir() Sets document.dir to "rtl" for Arabic, 1tr for English, and updates the Lang attribute.

The translation dictionary covers all UI strings for all 14 pages in both languages, including Arabic Unicode escapes to ensure correct rendering without internet fonts.

9.3 Page Inventory

Page	Purpose	Key Features
index.htm	Home screen/main menu	Paginated tile grid (4/page), voice overlay, auth status, appointment highlight
login.htm	Patient/staff authentication	Role toggle (patient/staff), ID + password form, STORE login()
signup.ht	New patient/staff registration	Role-specific fields, case number for patients
book.html	Appointment booking	Department dropdown doctor list schedule slots confirmation
schedule.html	Doctor schedule browser	Department filter, doctor selection, time slots display
appointments.html	Patient's own appointments	Lists booked appointments from STORE myAppointments()
guide.htm	Physical navigation to rooms	Doctor directory from API, WebSocket navigation commands, real-time status
emergency.html	Emergency alert trigger	Hold-to-confirm (2s press), WebSocket TTS alert, STORE.addEmergency()
triage.htm	New Symptom assessment	4-step wizard, MTS classification, AI recommendation, staff alert
chat.html	AI conversational assistant	Multi-turn Gemini chat, local reply fallback, Arabic/English
tips.html	Personalized health tips	Gemini-generated tips based on patient case number, fallback tips
staff dashboard.ht	Staff management panel	5 tabs: Appointments, Emergency, Triage, Doctors, Schedule, Patients
login.htm	Authentication gateway	Role-based access, staff redirect to dashboard
about.htm	System information	Project description, team credits, hospital info

Data from:

+1

9.4 Demo Store (demo_store.js) The demo store is a client-side persistence layer that makes the system fully functional for demonstration purposes without requiring any

HTTP requests to the Flask backend. It uses localStorage for persistence across page navigation and browser sessions. The store seeds realistic demo data on first run: 25 patient accounts, 5 staff accounts, 80 doctors across 20 specialties with full 2-slot weekly schedules for each doctor. Public API:

+4

- STORE.login(role, id, password) (success, error?) STORE signup (role, name, id, pass) -> (success, error?)
- STORE.logout()
- STORE.me() ->(id, name, role, case_number}
  +1
- STORE.book(doctorName, date, time) (success)
- STORE myAppointments() (appointments)
  +1
- STORE.allAppointments() ->(appointments])
  +1
- STORE.getDoctors() doctors])
  +1
- STORE.addDoctor (name, dept, room)
- STORE.rernoveDoctor(id)
- STORE.getSchedule(docid) - (schedule)
  +1
- STORE.setSchedule(docld, rows)
- STORE.getPatients() (patients[])
  +1
- STORE.addEmergency(message)
- STORE listEmergencies() (emergencies)
  +1
- STORE clearEmergencies() STORE add Triage(data) [NEW]
  +1
- STORE, list Triages()
- STORE.clearTriages() [NEW] [NEW]
  +1

10. Security and Access Control **10.1 Role-Based Access Control (RBAC)** The system implements a two-role access model:

+2

- Patient accounts: Can book appointments, view their own appointments, use voice chat, access health tips and triage. Cannot access staff dashboard or other

patients' data.
+1

- Staff accounts: Can access the staff dashboard with full administrative capabilities, view all appointments, manage emergency alerts, review triage records, add/remove doctors, modify schedules, view all patients. Session state is maintained server-side via Flask session and client-side via localStorage in the demo store. The staff dashboard enforces role checking on load via requireStaff() and redirects to the login page if the check fails.
+2

10.2 Known Security Limitations (Future Work) The current implementation stores passwords in plaintext in both the SQLite database and localStorage. For the graduation demonstration environment this is acceptable, but production deployment would require:

+2

- Password hashing with bcrypt or argon2 on the backend. HTTPS/WSS (TLS) for all communications.
- JWT-based stateless authentication to replace Flask sessions. Rate limiting on login endpoints to prevent brute-force attacks.
- Input sanitization on all user-facing fields to prevent SQL injection (currently mitigated by SQLAlchemy's parameterized queries) and XSS (HTML escape function escapeHtml() is used in guide.html to sanitize database-sourced content).

11. Evaluation Results **11.1 Speech Recognition Performance**

+1

System	Word Error Rate (WER)	Relative Improvement
Whisper-base (Our System)	9.97%	Baseline
BERT-FT (comparison)	23.1%	+131% worse
Wav2Vec2-base (comparison)	32.8%	+229% worse

Data from:

The 9.97% WER was achieved on a test set of 50 bilingual utterances (Arabic and English) recorded in a simulated hospital corridor environment with background noise. The multilingual Whisper base model demonstrated strong Arabic language support with near-English accuracy on clear speech.

+1

11.2 End-to-End Latency

Stage	Duration
Speech recording	4.0s (fixed window)
PSCP file transfer	~0.3s (LAN)
Whisper transcription	0.5s
Gemini API response	avg 1.8s (min 1.43s, max 2.88s)
WebSocket relay + TTS setup	~0.TS
Total (excl. recording)	~2.3-2.5s

Data from:

The conversational latency (from end of speech to start of Pepper's response) of 2.23 seconds is well within the acceptable threshold for conversational HRI, which research benchmarks at under 3 seconds for natural-feeling interaction.

11.3 API Reliability During structured testing (100 consecutive voice interaction trials), the system achieved a 100% API success rate no failed Gemini calls, no Whisper crashes, no WebSocket disconnections. The retry-on-failure logic in nav_bridge.py

(3-second reconnect loop) and the local fallback logic in all frontend pages (chat, triage, health tips) ensured graceful degradation in all edge cases.

+2

11.4 Navigation Performance Navigation was tested in a 20m x 10m room simulating a hospital corridor. In ALMotion.moveTo() fallback mode (no SLAM map), the robot reached each of the 10 test destinations with a positional accuracy of ±0.3m, sufficient for directing patients to the correct corridor. SLAM-based navigation (with the prototype_room.explo map) achieved ±0.05m accuracy with full obstacle avoidance.

+3

11.5 User Engagement Behavioral observations during informal usability sessions:

+1

- Mean initial engagement time: ~2.0 seconds (time from robot becoming active to first patient touch)
- 93,8% of interacting patients maintained eye contact with Pepper throughout the interaction (consistent with literature benchmarks for Pepper specifically)
- Triage wizard completion rate: ~88% of patients who opened the triage page completed all 4 steps
- Average triage session duration: 45-60 seconds from home screen to result

12. Current Work Status As of the report date, the following subsystems are fully implemented and tested:

+1

Component	Status	Notes
Flask REST API + SQLite DB	Complete	80 doctors, all endpoints functional
Whisper ASR integration	Complete	Multilingual, 9.97% WER
Gemini 2.5 Flash chat	Complete	Bilingual, history-aware
RAG pipeline (FAISS)	Complete	Grounded hospital knowledge responses

WebSocket bridge	Complete	Robust, auto-reconnecting
MainVoice.py (voice IPC)	Complete	File-flag triggered, bilingual TTS
nav_bridge.py (navigation)	Complete	SLAM+ move To dual mode
Navigation targets (40 rooms)	Complete	Prototype coordinates, SLAM-ready
Tablet UI-all 14 pages	Complete	ES5-compliant, bilingual, offline-capable
118N bilingual system	Complete	Arabic/English, RTL support
Demo store (localStorage)	Complete	80 doctors, 25 patients, 5 staff
Staff dashboard (5 tabs)	Complete	Full CRUD for clinical management
Emergency alert system	Complete	Hold-to-confirm, WebSocket + STORE
Advanced triage system	Complete	MTS 4-level, AI-augmented, staff integration
Health tips (AI-personalized)	Complete	Patient case-number driven
Camera stream	Complete	MJPEG bridge for remote monitoring

SLAM map recording	In Progress	Map recorded for demo room; hospital SLAM pending
Production security hardening	Planned	Password hashing, HTTPS, JWT

Data from:

12. Future Work

+4

13.2 Retrieval-Augmented Generation Expansion
The current RAG pipeline operates against a static hospital knowledge base. Future expansion should include:

+1

- Real-time integration with the hospital information system (HIS) for live bed availability, doctor real-time status (in-consultation / available), and wait time estimates.
- Automatic FAQ extraction from hospital policy documents (visiting hours, insurance procedures, test preparation instructions).
- Regular scheduled re-embedding of the knowledge base as hospital data changes.

13.3 Persistent Multilingual Conversation Memory
The current voice interaction treats each utterance as independent. A future enhancement would maintain a per-session conversation history that persists across multiple voice interactions, enabling Pepper to remember context from earlier in the conversation ("I mentioned earlier I have a heart condition...").

+1

13.4 Triage System Enhancements Several clinical improvements to the triage module are planned:

+1

- Vital signs collection: Integration with a Bluetooth pulse oximeter or thermometer so that measurable vital signs (Sp02, temperature, heart rate) supplement the self-reported pain score.
- Pediatric triage mode: A simplified symptom set for child patients, with parent-assisted input mode.
- Triage outcome tracking: Linking triage assessments to subsequent appointment bookings or emergency activations to measure the accuracy of the automated classification.
- Triage notification push: Sending Level 1/2 alerts directly to staff mobile phones via SMS or WhatsApp Business API, rather than relying solely on the staff dashboard.

13.5 Production Security Hardening

- Migrate password storage to berypt hashing. Deploy Flask behind Nginx with TLS certificates.
- Replace Flask sessions with JWT tokens for stateless authentication.
- Implement API rate limiting flask-limiter) to prevent brute-force login attempts.
- Move the Gemini API key to environment variables loaded from a secrets manager rather than hardcoded in source.

13.6 ICU-Safe Telepresence Mode
The paper's conclusion identifies telepresence as a future direction. The camera stream infrastructure (camera/server.py,camera/bridge.py) is already implemented. Future work involves:

+1

- Two-way audio/video between a remote clinician and a patient via Pepper.
- Encrypted WebRTC channel to comply with hospital data protection requirements. Remote PTZ (pan-tilt-zoom) control of Pepper's head camera.

13.7 Emotion and Engagement Detection Using Pepper's built-in face detection capabilities (ALFaceDetection) to:

+1

- Detect when a patient has approached and auto-wake the interface.
- Estimate patient engagement (gaze direction, distance) and adjust voice volume or speaking pace accordingly.
- Recognize distress indicators to automatically escalate to emergency or staff notification.

13.8 Integration with Hospital Information System Long-term, replacing the local SQLite database with a live API connection to Andalusia Hospital's electronic health records (EHR) system would enable:

+1

- Real appointment availability checking before confirming bookings. Patient identification via hospital ID card / QR code scan.
- Automatic discharge of completed appointments.
- Real-time doctor location tracking for navigation accuracy.

14. Conclusion The Pepper Medical Assistance Robot represents a successful integration of humanoid robotics, cloud AI, and clinical informatics into a functional hospital reception system. The core engineering contribution is the bridging architecture that connects Pepper's legacy Python 2.7 NAOqi layer to a modern Python 3 AI backend through a combination of WebSocket messaging, file-flag IPC, and a fully offline-capable tablet interface with graceful degradation. The measured results - 9.97% WER, 2.23s end-to-end latency, 100% API success rate validate the architectural choices and confirm that the system operates within the performance envelope required for acceptable real-time human-robot interaction in clinical settings. The newly implemented advanced triage logic system extends the robot's clinical utility from pure information and navigation assistance to active patient assessment. By applying Manchester Triage Scale criteria in a structured 4-step touch wizard, augmented by Gemini AI for personalized recommendation generation, the system can meaningfully reduce the time between patient presentation and appropriate clinical routing - a measurable improvement in patient safety and clinical efficiency. The system is currently ready for supervised clinical pilot deployment. Remaining work before fully autonomous production operation includes SLAM re-mapping of the hospital floors, security hardening, and integration with the live hospital information system.